

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет о командном программном проекте на тему:
Разработка онлайн бизнес-игры “Мультикомандный Featureban”
(промежуточный, этап 1)

Выполнили студенты:

группы #БПМИ244, 2 курса	Бакунц Арина Александровна
группы #БПМИ245, 2 курса	Денисов Николай Дмитриевич
группы #БПМИ246, 2 курса	Овчаров Иван Евгеньевич
группы #БПМИ249, 2 курса	Мостова Александра Александровна
группы #БПМИ249, 2 курса	Сухобок Тимофей Михайлович
группы #БПМИ2411, 2 курса	Чупрына Матвей Михайлович

Принял руководитель проекта:

Коротков Александр Олегович
Руководитель группы, группа дополнительных продуктов
АО "Т-Банк"

Содержание

1	Аннотация	3
2	Введение	4
3	Обзор литературы	6
4	Процессы разработки	9
4.1	Сравнительный анализ методологий управления разработкой	9
4.2	Обоснование гибридного подхода: Асинхронный пайплайн	9
4.3	Этапы реализации стори в рамках процесса	10
4.4	Визуализация и контроль потока	10
5	План дальнейшей работы	12
5.1	Этапы реализации проекта	12
5.2	Функциональная декомпозиция системы	13
5.3	Распределение задач по участникам	13
6	Технические документации	14
6.1	Проектирование инфраструктуры и регламентация процессов управления исходным кодом	14
6.1.1	Организация репозитория	14
6.1.2	Проектирование инфраструктуры и регламентация процессов управления исходным кодом	15
6.1.3	Стратегия управления ветвлением	15
6.1.4	Технологический стек и автоматизация инфраструктуры	16
6.1.5	Оптимизация кросс-репозиторного взаимодействия	17
6.1.6	Процесс релиза	17
	Список литературы	18

1 Аннотация

Проект посвящен разработке веб-версии бизнес-игры «Мультикомандный Featureban» — инструмента для проведения дистанционных корпоративных тренингов, направленных на развитие навыков межкомандного взаимодействия и совершенствование коммуникации в условиях совместной работы над проектами. Эта игра помогает просимулировать популярный в данный момент в крупных IT-компаниях метод разработки - Kanban [12].

Классическая очная версия игры давно зарекомендовала себя как эффективный способ выявления и разрешения проблем координации и синхронизации действий между разными подразделениями при разработке одного продукта. Однако её очный формат существенно ограничивает масштабируемость применения: требует физического присутствия участников (а также помещение для проведения тренинга), усложняет работу модератора и создает барьеры в виде географических ограничений.

Онлайн-версия устранил эти ограничения, позволяя проводить тренинги в полностью дистанционном формате, упростив процесс проведения данной игры и обеспечив возможность одновременного участия команд, расположенных в разных регионах. Приложение будет предоставлять игрокам интуитивный интерфейс для взаимодействия и отслеживания игровых метрик, а также специализированный модуль для тренера, позволяющий контролировать ход игровой сессии и анализировать результаты .

Планируемые результаты: Разработано полнофункциональное веб-приложение с архитектурой, обеспечивающей стабильное взаимодействие множества пользователей в реальном времени. Система включает интерфейс для участников (с визуализацией процессов, отображением карточек и метрик), модуль управления командами и специализированный инструмент для тренера.

Ключевые слова

Канбан, симуляция, бизнес-игра, Фичебан, мультикомандный, корпоративные тренинги, веб-приложение, закон Литтла, Телеграм бот

2 Введение

В условиях современного развития информационных технологий и усложнения программных продуктов существенно возрастает сложность процессов их разработки и сопровождения. Современные компании, особенно в сфере информационных технологий, всё чаще переходят от работы отдельных автономных команд к модели, в которой несколько команд параллельно развивают один продукт или платформу. Такая организация труда требует более высокого уровня координации, прозрачности процессов и согласованности действий между участниками.

В этих условиях особое значение приобретают гибкие методологии управления проектами, ориентированные не только на локальную эффективность отдельных команд, но и на оптимизацию всей системы в целом. Одной из наиболее распространённых и практически применяемых методологий является Kanban. В отличие от итеративных подходов, таких как Scrum, Kanban ориентирован на непрерывный поток работ, визуализацию процессов и ограничение количества незавершённых задач. Данный подход хорошо масштабируется и широко используется в организациях со сложной структурой и уже существующими продуктами.

При этом ключевые принципы Kanban, включая ограничение WIP (Work in Progress), управление потоком и анализ метрик производительности, достаточно сложно усваиваются при изучении исключительно теоретических материалов. Без практического опыта участникам трудно осознать влияние управленческих решений на пропускную способность системы, время выполнения задач и стабильность процессов. В результате между формальным знанием методологии и её реальным применением на практике возникает существенный разрыв.

Для преодоления данного разрыва в корпоративном обучении и образовательных программах широко применяются бизнес-игры и игровые симуляции. Они позволяют участникам в безопасной среде экспериментировать с процессами, принимать управленческие решения и наблюдать их последствия без риска для реальных проектов. Игровой формат способствует более глубокому пониманию системных эффектов, возникающих при управлении потоком работ, и позволяет наглядно продемонстрировать влияние ограничений и изменений в процессе разработки.

Одной из таких симуляций является игра Featureban, моделирующая процесс разработки продукта с использованием Kanban-подхода. Классическая версия игры ориентирована на работу одной команды и позволяет продемонстрировать базовые принципы управления потоком. Мультикомандная версия Featureban представляет собой развитие данной симуляции и направлена на моделирование взаимодействия нескольких команд, одновременно

работающих над общим продуктом. В рамках такой игры проявляются межкомандные зависимости, узкие места системы, проблемы синхронизации и эффекты локальной оптимизации, характерные для реальных крупных проектов.

Актуальность данной работы обусловлена возрастающей сложностью управления процессами разработки в условиях масштабирования продуктов и увеличения количества команд, вовлечённых в совместную работу. На практике многие организации сталкиваются с тем, что методы и практики Kanban, успешно применяемые на уровне одной команды, теряют эффективность при переходе к мультикомандному формату. При этом существующие инструменты и обучающие материалы не позволяют в полной мере продемонстрировать системные эффекты, возникающие при межкомандном взаимодействии, и обеспечить корректное воспроизведение игровой механики мультикомандной симуляции.

В этой связи актуальной является задача создания специализированного программного инструмента, позволяющего наглядно моделировать мультикомандные процессы разработки, демонстрировать влияние управленческих решений на всю систему в целом и обеспечивать автоматизацию игровых правил и сбора метрик. Решение данной задачи способствует повышению эффективности обучения и более осознанному внедрению Kanban-подходов в организациях со сложной структурой.

Целью данной курсовой работы является разработка онлайн-версии бизнес-игры «Мультикомандный Featureban», предназначенной для моделирования межкомандного взаимодействия и использования в обучающих и тренинговых форматах. Для достижения поставленной цели в рамках работы проводится анализ предметной области, формулируются требования к разрабатываемой системе, проектируется архитектура веб приложения и реализуются основные компоненты системы.

В результате планируется создать веб приложение, поддерживающее проведение игровых сессий с участием нескольких команд, разграничение ролей пользователей, визуализацию процессов и автоматизированный сбор ключевых метрик игрового процесса. Новизна данной работы заключается в разработке специализированного инструмента для онлайн-проведения мультикомандной версии Featureban, который учитывает особенности игровой механики, поддерживает сценарии симуляции и обеспечивает централизованное управление игровым процессом. Практическая значимость работы состоит в возможности использования разработанного приложения для проведения корпоративных тренингов и образовательных мероприятий, направленных на изучение принципов управления потоком и межкомандного взаимодействия.

3 Обзор литературы

Среди гибких методов управления самыми популярными являются Scrum и Kanban. Идея методологии Scrum - разработка итерациями. То есть за определенный промежуток времени команда должна выпустить кусок готового продукта. Данная методология часто используется в условиях разработки нового продукта, однако в крупных компаниях с большим количеством команд и уже сделанным большим продуктом широко используется второй метод - Kanban. Он сфокусирован на управлении потоками работ и ограничении незавершенных работ.

Принципы гибких методов управления сложно усвоить исключительно с помощью книг и статей. Эта проблема подтверждается результатами исследований в области менеджмента и бизнес-образования. Например, в работе *The effectiveness of games for educational purposes: A review of recent research* [2] показано, что игровые методы обучения и симуляции оказываются более эффективными при формировании сложных управленческих навыков по сравнению с традиционными форматами, такими как проведение лекций или прочтение соответствующей литературы. Такие умения требуют понимания как работают процессы внутри системы и то как эти процессы влияют друг на друга в рамках этой системы, и, как подчеркивают сами авторы, возможность принимать решения в игровой среде, а также повышенная вовлечённость участников процесса позволяют овладеть этими навыками эффективнее и усвоить материал лучше.

Это особенно актуально для подходов, таких как Kanban [12] и Lean [11]. Поэтому в обучении широко используются игровые симуляции. С помощью таких симуляций участники экспериментируют с процессами и наблюдают последствия решений в безопасной игровой среде. Эффективность этого подхода подтверждается теорией описанной в книге Д. Колба (*Experiential Learning Cycle*) [1]. Согласно модели Д. Колба, глубокое обучение происходит через цикл из четырех этапов: получение конкретного опыта, рефлексивное наблюдение, абстрактная концептуализация и активное экспериментирование. Участие в бизнес-игре позволяет людям быстро пройти этот цикл, получая ценный практический опыт. Кроме того, исследования эффективности бизнес-симуляций, такие как работа *Learning with a strategic management simulation game: A case study* [9], показывают, что игровой формат способствует развитию системного мышления. Авторы отмечают, что симуляции позволяют связать теоретические знания с практическими навыками. Подобные форматы обучения дают возможность участникам увидеть взаимосвязи между различными элементами внутри сложной системы, что сложно или недостижимо при традиционных методах обучения.

Рассмотрим подробнее эти подходы. Метод Kanban [3], описанный Дэвидом Андерсоном, формулирует шесть основных практик Kanban:

- 1 Визуализация рабочего процесса
- 2 Ограничение WIP (Work in Progress)
- 3 Управление потоком
- 4 Явные правила выполнения работ (explicit policies)
- 5 Внедрение обратной связи через совместные встречи
- 6 Совершенствование на основе модели (collaborative improvement)

а также подчеркивает принцип: *start with what you do now*. Для обучающей симуляции наиболее важными являются практики: визуализация рабочего процесса, ограничение WIP, управление потоком. В игровой форме именно эти практики позволяют участникам наглядно видеть связь принятых решений в управлении и скорость доведения задач до конца. Этот метод тесно связан с концепцией бережливого производства (Lean). Оба подхода направлены на устранение потерь и увеличение пропускной способности. Эти практики и принципы опираются на теорию ограничений Э. Голдратта [7], которая утверждает, что производительность всей системы определяется местами с наибольшим количеством ограничений. В совокупности рассмотренные подходы формируют основу для построения игровой симуляции, которая моделирует реальные рабочие процессы с целью обучения.

В области управления проектами руководители широко применяют закон Литтла [13] для расчета производительности бизнеса и для повышения прогнозируемости своих рабочих процессов. В методологии Kanban закон выражается формулой:

$$L = \lambda \times W \quad (1)$$

где L - это незавершенная работа (количество активных задач в системе), λ - пропускная способность (количество выполненных задач за единицу времени), W - время цикла (время прохождения задачи через процесс). Однако, прочитав, сложно понять, как она работает на самом деле, людьми эта формула воспринимается абстрактно. Игровая симуляция дает людям понять на примере, как действует этот закон, и позволяет увидеть, какой результат дают ограничения на WIP.

На данный момент есть онлайн-версия игры Featureban¹, но эта версия игры только для одной команды и рассчитана на 3–5 человек. Версия игры на несколько команд существует только в оффлайн формате и разработана сертифицированными экспертами из России (Neogenda)[14]. Оригинальная игра Featureban, разработанная Майком Барроузом, распространяется по лицензии Creative Commons Attribution ShareAlike 4.0 [4], соответственно все материалы по мультикомандному Featureban распространяются по той же лицензии.

В настоящее время единственный способ провести игру на несколько команд - это использовать универсальные инструменты, такие как онлайн-доску Miro или сервис для управления проектами Trello. Однако данные решения не могут быть напрямую использованы для реализации симуляции.

Во-первых, они только визуализируют игру, но не накладывают никаких ограничений на действия участников. Из-за этого пропадает поддержка ролей, разделение на команды, разрушаются четкие сценарии игр, что являются критически важными аспектами симуляции для корректного моделирования принципов Kanban. Исследования игровых симуляций подчеркивают, что строгость правил, ограничений и сценариев является критически важным аспектом подобных обучающих методов.

Во-вторых, подобные платформы не собирают нужную статистику. При отсутствии автоматизации сбора статистики увеличивается влияние человеческого фактора, становится сложнее проводить рефлекссию над экспериментом, что приводит к снижению педагогической ценности проведения этой игры.

Таким образом, такие универсальные инструменты не предоставляют достаточно функциональный интерфейс для высокоэффективного проведения мультикомандной версии игры.

¹Онлайн версия игры доступна по ссылке: <https://tools.flowfast.io/featureban>, дата обр. 13.01.2026

4 Процессы разработки

Выбор методологии управления проектом является критическим фактором для начинающей команды. В данном разделе проводится сравнительный анализ существующих подходов и приводится научное обоснование выбранного гибридного процесса разработки онлайн-симулятора "Мультикомандный Featureban".

4.1 Сравнительный анализ методологий управления разработкой

Для определения оптимального пути реализации проекта были рассмотрены три доминирующих парадигмы в индустрии программного обеспечения:

- **Каскадная модель (Waterfall):** Линейный подход, требующий полной фиксации требований до начала написания кода. Несмотря на предсказуемость, метод был отвергнут ввиду высокой степени неопределенности продукта. В образовательных симуляторах критически важна возможность корректировки логики игры на основе первых тестов, что невозможно в жестких рамках Waterfall.
- **Scrum-фреймворк [10]:** Итеративный подход, ориентированный на кросс-функциональность. Команда отказалась от "чистого" Scrum, так как в условиях становления команды и наличия ярко выраженной специализации (выделенная роль аналитика) возникал риск неэффективного планирования и простоя разработчиков в ожидании документации в начале спринта.
- **Kanban-метод [3]:** Система управления потоком, минимизирующая незавершенное производство (WIP). Несмотря на использование элементов Kanban в самой тематике нашего продукта, для внутренней разработки метод показался недостаточно структурированным для новой команды, которой необходим спринты для поддержания темпа.

4.2 Обоснование гибридного подхода: Асинхронный пайплайн

В результате анализа был сформирован гибридный метод, сочетающий дисциплину Scrum (итерации) и принципы управления потоком. Суть метода заключается в разделении жизненного цикла задачи на два параллельных трека:

Аналитика: Опережающая работа аналитика. Пока команда разработки занята текущей итерацией, аналитик готовит документацию, схемы API и требования для следующей "Стори". Это позволяет нивелировать риск "пустого бэклога" на старте итерации.

Разработка: Реализация согласованного объема задач разработчиками в фиксированные циклы от 1 до 3 недель. Данная схема научно обоснована Теорией ограничений Э. Голдратта [7]. Выделяя аналитику в отдельный этап, мы расширяем "узкое горлышко" системы, обеспечивая максимальную пропускную способность команды разработки.

4.3 Этапы реализации стори в рамках процесса

Процесс обработки каждой крупной функциональной единицы (Стори) в нашей команде жестко регламентирован и состоит из следующих фаз:

- 1 **Аналитика:** Проработка технического дизайна, ролевых моделей и API. Здесь происходит декомпозиция Стори на технические задачи.
- 2 **Синхронизация "Three Amigos":** Короткая сессия (созвон), на которой аналитик презентует документацию. Это критический этап контроля качества и "выравнивания" понимания между фронтенд, бэкенд-разработчиками и девопс инженерами.
- 3 **Итеративная реализация:** Разработка функционала, покрытие юнит-тестами (как фронтенд, так и бэкенд частей) и настройка мониторинга/аллертов.

Такое разделение позволяет следовать принципам "Lean Thinking-[11], устраняя потери, связанные с ожиданием уточнений и исправлением ошибок, возникших из-за неверной интерпретации требований.

4.4 Визуализация и контроль потока

Для оперативного управления разработкой и обеспечения прозрачности процессов была внедрена электронная канбан-доска. Ее структура отражает специфику выбранного гибридного метода и позволяет отслеживать движение каждой функциональной единицы через четыре ключевых этапа:

- 1 **Todo:** этап формирования и приоритезации бэклога.
- 2 **In progress:** активная фаза реализации. Для задач аналитики этот этап подразумевает исследование и создание документации. Для задач разработки - непосредственное написание кода на основе уже готовых требований.
- 3 **Review:** критический этап верификации. В контексте аналитики этот статус означает проведение сессии, на которой аналитик представляет свое решение. В контексте разработки — проведение перекрестного код-ревью и контроль качества.

4 **Done:** финальное состояние задачи, подтверждающее её соответствие критериям готовности (Definition of Done) и готовность к следующему циклу или релизу.

Особое внимание уделяется правилам перехода между фазами. Большая задача перемещается в статус "In progress" для команды разработки только после того, как соответствующая ей задача на аналитику перешла в статус "Done". Такая дисциплина исключает старт работ по неполным спецификациям и минимизирует нагрузку на программистов, обеспечивая стабильный поток создания ценности в соответствии с законом Литтла [13] и принципами теории ограничений [7].

Все процессы были рассмотрены и приняты командой как стандарт взаимодействия.

5 План дальнейшей работы

5.1 Этапы реализации проекта

В рамках курсового проекта будет применён метод итерационной разработки, предполагающий поэтапное уточнение требований и постепенное наращивание функциональности. Первоначально планируется выбор технологического стека и реализация минимального жизнеспособного продукта (MVP — Minimum Viable Product), содержащего базовый функционал проекта:

- Главное меню игры с возможностью сформировать команды и запустить игру.
- Командные и общие доски.
- Карточки проектов и задач с функционалом блокировки и перетаскивания по доске.
- Логика "подбрасывания монетки" и хода игрока.
- Механизм смены команды игроком в процессе сессии.
- Интерфейс для фасилитатора (ведущего) игры.
- Генерация карточек с проектами.
- Поддержка одной игровой сессии.
- Развёртывание базовой инфраструктуры для функционирования игровой системы.

Затем в последующих итерациях система будет дополняться расширенными возможностями и оптимизациями, среди которых:

- Обработка случаев потери сетевого подключения игроком.
- Повышение отказоустойчивости и надёжности системы.
- Организация фазы ретроспективного обсуждения между игроками.
- Создать правила, по которым игрок переходит в другую команду.
- Сбор метрик и статистики для дальнейшего анализа производительности.
- Доработка пользовательского интерфейса (UI/UX), улучшение визуального дизайна и анимационных решений.

5.2 Функциональная декомпозиция системы

Проектируемая система включает следующие основные функциональные составляющие:

- **Игровая логика**

- сценарий проведения игры;
- этапы игры и переходы между ними;
- сбор и расчёт игровых метрик (WIP, lead time и др.).

- **Пользовательские роли**

- ведущий (модерация игры, управление процессом);
- команда (совместное принятие решений);
- игрок (участие в игровых активностях).

- **Сетевая и многокомандная часть**

- синхронизация состояния игры между участниками;
- поддержка нескольких команд в рамках одной сессии.

- **Пользовательский интерфейс**

- интерфейс ведущего;
- интерфейс команд и игроков.

- **Хранение и обработка данных**

- данные игровых сессий;
- результаты игр и статистика.

5.3 Распределение задач по участникам

Задачи будут распределяться по мере выполнения проекта. Работа планируется по методологии Канбан, в которой не предусмотрено привязки задач к конкретным исполнителям до момента их взятия в работу.

6 Технические документации

6.1 Проектирование инфраструктуры и регламентация процессов управления исходным кодом

6.1.1 Организация репозитория

Разработка системы ведётся в трёх отдельных репозиториях: бэкенд-репозитории, фронтенд-репозитории и инфраструктурном репозитории.

Бэкенд-репозиторий содержит серверную часть приложения: реализацию игровой логики и правил, серверный API, механизмы хранения и обработки состояния игровых сессий, управление пользователями и ролями, а также синхронизацию данных между клиентами в реальном времени.

Фронтенд-репозиторий содержит клиентскую часть приложения: пользовательский интерфейс для участников и ведущего, визуализацию состояния игры и метрик, а также логику взаимодействия с сервером через API.

Инфраструктурный репозиторий содержит конфигурационные файлы для развёртывания всей системы: конфигурацию веб-сервера Nginx, файлы Docker Compose для сборки локальных и production-окружений, скрипты деплоя и настройки CI/CD-пайплайна.

На начальном этапе проектирования рассматривалась классическая двухрепозитория модель: один репозиторий для бэкенда и один для фронтенда. Это базовое решение не требует дополнительных усилий по организации и является распространённой практикой для небольших проектов.

По мере проектирования инфраструктуры возник вопрос о размещении файлов, не относящихся ни к логике бэкенда, ни к интерфейсу фронтенда: конфигурации Nginx, файлов Docker Compose, объединяющих оба сервиса, и скриптов деплоя. Рассматривались следующие варианты:

- 1 **Разместить инфраструктурные файлы в бэкенд-репозитории:** отклонено, поскольку нарушает принцип единственной ответственности и смешивает бизнес-логику с инфраструктурными конфигурациями. Фронтенд-команда теряет прямой контроль над параметрами деплоя.
- 2 **Разместить инфраструктурные файлы во фронтенд-репозитории:** отклонено по аналогичным причинам.
- 3 **Выделить инфраструктурные файлы в отдельный репозиторий:** принято.

Инфраструктурные конфигурации выделены в отдельный репозиторий.

6.1.2 Проектирование инфраструктуры и регламентация процессов управления исходным кодом

Для обеспечения стабильного темпа разработки и минимизации технологических потерь была спроектирована специализированная инфраструктура и стратегия ветвления. В основу легли принципы изоляции компонентов, автоматизации рутинных операций и строгого ограничения незавершенного производства (WIP).

6.1.3 Стратегия управления ветвлением

В ходе проектирования системы был проведен сравнительный анализ трех доминирующих стратегий управления ветвлением:

- **Git Flow** [5]: была отвергнута ввиду избыточной сложности для небольшой команды. Наличие долгоживущих веток `develop` и `release` создает дополнительные издержки и усложняет процесс интеграции готового кода в проект, что противоречит принципам минимизации потерь.
- **Trunk-Based Development** [8]: метод предполагает интеграцию малых порций кода напрямую в `master`. Данный подход был признан рискованным на текущем этапе, так как он требует предельно высокого покрытия автоматизированными тестами и высокой дисциплины, отсутствие которых могло бы привести к дестабилизации основной ветки.
- **GitHub Flow** [6]: данная модель наиболее близка к потребностям команды, однако в "чистом" виде она не обеспечивает необходимый уровень контроля над готовностью крупных функциональных блоков.

В результате была разработана авторская адаптация, которая интегрирует простоту GitHub Flow с жестким контролем WIP-лимитов. Выбор данной стратегии обусловлен необходимостью гарантировать поставку только полностью завершенной ценности. Процесс строится на следующих правилах:

- 1 Ветка `master` в обоих репозиториях (Frontend и Backend) является абсолютно стабильной и всегда содержит код, готовый к эксплуатации. Любое слияние в `master` автоматически инициирует процесс деплоя на сервер.

- 2 Для каждой крупной задачи создается отдельная ветка `story/название-задачи`. В отличие от GitHub Flow, где ветки создаются под мелкие фичи, наш подход предполагает накопление всех связанных с одной Стори изменений в одной ветке. Все атомарные технические задачи, декомпозированные на этапе аналитики, вливаются именно в эту ветку через локальные Pull Requests.
- 3 Слияние ветки `story` в `master` происходит только тогда, когда все задачи данной стори в рамках конкретного репозитория выполнены, прошли код-ревью и комплексное тестирование. Это исключает ситуацию, когда в основную ветку попадает частично реализованный функционал.
- 4 Команда следует принципу однопредметного потока из теории бережливого производства. Разработка новой Стори не начинается до момента полного завершения и слияния текущей в `master`.

Обоснование выбора данной модели базируется на Законе Литтла [13]: ограничивая количество одновременно открытых веток крупных задач до одной, команда минимизирует объем незавершенной работы. Это позволяет сократить время прохождения задачи по циклу и повысить предсказуемость релизов. Такой подход к управлению ветвлением помогает поддерживать высокий уровень архитектурного качества и предотвращает накопление технического долга в основной ветке разработки.

6.1.4 Технологический стек и автоматизация инфраструктуры

Для реализации выбранной стратегии был применён набор технологий, который обеспечивает наилучшее соответствие между различными средами.

- **Контейнеризация (Docker):** Использование Docker позволяет инкапсулировать зависимости приложения в изолированные контейнеры. Это решает проблему несовместимости библиотек на разных машинах разработчиков.
- **Реестр образов (Container Registry):** Для координации между бэкенд- и фронтенд-разработчиками используется механизм публикации артефактов (образов).
- **Веб-сервер Nginx:** Выполняет роль обратного прокси-сервера на единственном сервере проекта, обеспечивая маршрутизацию трафика и безопасность.

Использование одного сервера на текущем этапе оправдано отсутствием необходимости в горизонтальном масштабировании при низкой начальной нагрузке. Это позволяет избежать "избыточной обработки" одного из видов потерь в Leap.

6.1.5 Оптимизация кросс-репозиторного взаимодействия

Особое внимание уделено удобству разработки. Была проанализирована проблема зависимости фронтенд-разработчика от бэкенд-окружения. Рассмотренные варианты:

- 1 Полное дублирование: Фронтенд-разработчик клонирует оба репозитория. (Отвергнуто из-за сложности настройки и когнитивной нагрузки).
- 2 Mock-серверы: Использование заглушек вместо реального бэкенда. (Отвергнуто из-за риска интеграционных ошибок).

Выбранное решение - Разработка через Docker Registry. Процесс для фронтенд-разработчика выглядит следующим образом:

- 1 CI/CD пайплайн бэкенда при каждом обновлении стабильной ветки автоматически собирает Docker-образ и пушит его в GitHub Container Registry с тегом :latest.
- 2 Фронтенд-разработчик в своем локальном окружении выполняет единую команду `docker-compose up`.
- 3 Docker автоматически загружает последний стабильный образ бэкенда из облачного реестра.

6.1.6 Процесс релиза

Релиз цикла фиксируется навешиванием Git-тега с семантической версией на последний коммит в ветке `master`. Это создает неизменяемую точку в истории проекта, к которой можно откатиться в случае обнаружения критических ошибок, обеспечивая устойчивость системы согласно принципам Agile.

Список литературы

- [1] Kolb D. A. *Experiential learning: Experience as the source of learning and development*. FT Press, 2014.
- [2] Randel J. M. et al. “The effectiveness of games for educational purposes: A review of recent research”. B: *Simulation & Gaming*. (1992).
- [3] David J. Anderson. *Kanban : successful evolutionary change in your software business*. Blue Hole Press, 2010.
- [4] *Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)*. URL: <https://creativecommons.org/licenses/by-sa/4.0/> (дата обр. 03.02.2026).
- [5] Vincent Driessen. *A successful Git branching model*. 2010. URL: <https://nvie.com/posts/a-successful-git-branching-model/> (дата обр. 10.03.2026).
- [6] GitHub. *Understanding the GitHub flow*. 2023. URL: <https://docs.github.com/en/get-started/using-github/github-flow> (дата обр. 10.03.2026).
- [7] Cox J. Goldratt E. M. *The goal: a process of ongoing improvement*. Routledge, 2016.
- [8] Paul Hammant. *Trunk-Based Development*. 2013. URL: <https://trunkbaseddevelopment.com/> (дата обр. 10.03.2026).
- [9] Kerridge C. Loon M. Evans J. “Learning with a strategic management simulation game: A case study”. B: *The International Journal of Management Education*. (2015).
- [10] Sutherland J. Schwaber K. *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. Scrum Alliance, 2020.
- [11] Jones D. T. Womack J. P. *Lean thinking: banish waste and create wealth in your corporation*. New York: Free Press, 2003.
- [12] Майк Барроуз. *Канбан Метод: улучшение системы управления*. Альпина паблишер, 2023.
- [13] *Малоизвестные факты о Законе Литтла: как обеспечить предсказуемость рабочего процесса*. URL: <https://neogenda.com/blog/little-law-prediction> (дата обр. 13.01.2026).
- [14] *Мультикомандный Featureban*. URL: <https://neogenda.com/multifeatureban> (дата обр. 13.01.2026).